The Banksys

ANSWER KEY (full solutions). Build an AI art-learning assistant in Python + Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

Welcome: what you are building . . . 3

How to use this tutorial . . . 3

The roadmap . . . 3

Before you start . . . 4

Set up your project . . . 4

Demo mode (no API key needed) . . . 5

The big picture . . . 5

Build it, step by step . . . 6

Step 1: Run the starter and read how it works . . . 6

Step 2: Feature 1, look up an art style . . . 8

Step 3: Feature 2, report the size of the upload . . . 10

Step 4: Feature 3, instructions for each improvement area . . . 11

Step 5: Feature 4, build the AI prompt . . . 13

Step 6: Feature 5, format the AI's answer . . . 15

Step 7: Turn on the real AI and test everything . . . 18

Step 8: Optional extensions . . . 19

What you learned . . . 19

Welcome: what you are building

The Banksys is a small web app that helps you learn about art. You will be able to:

1. Browse art styles (Impressionism, Cubism, Pop Art, Realism) and read about each one.
2. Upload your own drawing or painting.
3. Pick a **target style** and something you want to improve (shading, color, ...).
4. Get a friendly critique from an AI art teacher, split into **Strengths**, **Areas to Improve** and **Suggested Exercises**.

The folder you were given is a **scaffold**. The app already launches, but the important functions are empty and marked `TODO`. This tutorial walks you through every `TODO`, in the order you should do them. For each one you will find:

- **Goal**: what the function should do, in plain English.
- **Ideas you need**: the small bit of Python being practiced.
- **Do it**: step-by-step instructions with code.
- **Check it**: how to run it and what you should see.

> **TIP**
> Don't just copy and paste. Type the code yourself, and after each step run the app. The fastest way to learn is to break something, read the error, and fix it.

How to use this tutorial

- Keep this PDF open next to your editor. Work through the steps **in order**. Each step builds on the last.
- Code shown with a file name above it (for example `art_styles.py`) belongs in that file.
- Whenever you see **Checkpoint**, stop and run the app before moving on.
- If you get stuck, read the **Stuck?** boxes. They list the mistakes students make most often.

The roadmap

Step	File	What you build
1	`app.py`	Run the starter app and read how it works (no coding)
2	`art_styles.py`	Feature 1: look up an art style's description
3	`app.py`	Feature 2: report the size of an uploaded image
4	`feedback.py`	Feature 3: instructions for each improvement area
5	`feedback.py`	Feature 4: build the prompt that goes to the AI
6	`feedback.py`	Feature 5: format the AI's answer into three sections
7	(your `.env`)	Turn on the real AI and test the whole app
8	any	Optional extensions

Before you start

Set up your project

Open a terminal inside the `the_banksys` folder, then run the commands for your computer **one at a time**.

Open the terminal in the right folder:

- **Mac:** open the **Terminal** app, type `cd ` (with a space), drag the `the_banksys` folder onto the window, and press Enter. (Or right-click the folder in Finder and choose **New Terminal at Folder**.)
- **Windows:** open the `the_banksys` folder in File Explorer, click the address bar at the top, type `powershell`, and press Enter. A PowerShell window opens already inside the folder.
- **Either:** if you use VS Code, open the folder and choose **Terminal > New Terminal**.

On a Mac:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python app.py
```

On Windows (PowerShell or Command Prompt):

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python app.py
```

What each line does:

- `python -m venv venv` creates a private "box" of Python packages just for this project. (On a Mac the command is `python3` because many Macs have no plain `python` until the box is on.)
- `source venv/bin/activate` (Mac) or `venv\Scripts\activate` (Windows) steps into that box. You should now see `(venv)` at the start of your terminal line. **Do this every time you open a new terminal.**
- `pip install -r requirements.txt` installs the four packages the app needs (Gradio for the web page, Google GenAI for the AI, python-dotenv for reading your key, Pillow for images).
- `cp .env.example .env` (Mac) or `copy .env.example .env` (Windows) makes your private settings file. You can leave the key blank for now.
- `python app.py` starts the app. Open the address it prints (usually `http://127.0.0.1:7860`).

> **STUCK?**
> **Windows:** if `python` is "not found", try `py -m venv venv` instead (then use plain `python` once the box is on). If PowerShell says *running scripts is disabled on this system* when you activate, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer **Y**, and try again. Or just use **Command Prompt** instead of PowerShell. **Mac:** if `python3` is not found, install Python from python.org.

To stop the app, click the terminal and press `Ctrl + C`. **After you edit a Python file you must stop and restart the app** to see your change.

Demo mode (no API key needed)

The AI part needs a Google Gemini API key, which your teacher may give you later (or you can create one at Google AI Studio, aistudio.google.com/apikey). **Without a key the app runs in demo mode.** Everything works except the AI never looks at your picture; instead it prints a clearly labelled message plus **the prompt your code built**. That is perfect for checking your work, so you can finish Steps 1 to 6 without any key.

> **WARNING**
> Never paste your API key into a Python file, share it, or commit `.env` to GitHub. Keys go only in the `.env` file.

The big picture

When you click a button, information travels like this:

```
you click / upload in the browser
         |
         v
  app.py  (Gradio calls one of its functions)
         |
         +--> art_styles.py   look up a style description
         |
         +--> feedback.py     choose instructions -> build the prompt
                    |
                    v
               ai_helper.py    sends image + prompt to the AI, gets text back
                    |
                    v
               feedback.py     format the text nicely
                    |
                    v
  app.py shows the result in the browser
```

File	Job
`app.py`	The screen: tabs, buttons, and the functions that run when you click
`art_styles.py`	The dictionary of art styles and the lookup function
`feedback.py`	Turns your choices into a prompt, and the AI's answer into readable sections
`ai_helper.py`	The only file that talks to the AI. **You never need to edit it.**

You will edit `art_styles.py`, `feedback.py`, and one small piece of `app.py`.

Build it, step by step

Step 1: Run the starter and read how it works

Goal: understand the code that is already written, before you add to it. There is nothing to type in this step.

1a. Click around

Run `python app.py` and open the page. There are three tabs:

1. **Learn About Styles**: pick a style. Right now the description box says `TODO: implement get_style_description() in art_styles.py`. That is your first job.
2. **Upload Artwork**: drop in any image. It says "Artwork received!" but nothing else.
3. **Get Feedback**: pick a style and a focus area and click the button. You will see a demo-mode message.

1b. Read `app.py`

Open `app.py` and find the three parts of every Gradio app.

Part 1: handler functions. These are ordinary Python functions. Gradio calls them when something happens in the browser. Here is the first one:

```
def show_style(style_name):
    return get_style_description(style_name)
```

Gradio passes the dropdown's current value in as `style_name`. Whatever the function **returns** is displayed in an output component.

Part 2: components. These are the buttons, dropdowns and text boxes on the screen.

```
style_dropdown = gr.Dropdown(choices=get_style_names(), label="Choose a style")
style_output = gr.Textbox(label="Description", lines=4)
```

Part 3: wiring. This line connects them:

```
style_dropdown.change(show_style, inputs=style_dropdown, outputs=style_output)
```

Read it aloud: *"When the dropdown changes, call `show_style`, give it the dropdown's value, and put the result in `style_output`."* Every interactive thing in this app is built from that one pattern: **event, function, inputs, outputs.**

> **CONCEPT**
> The components listed in `inputs=[...]` are matched to your function's parameters by **position**, not by name. In `critique_button.click(handle_critique, inputs=[image_input, target_style, area_choice], ...)`, the first input becomes the first parameter `image`, the second becomes `style`, and the third becomes `area`. If you ever mix up the order, your function receives the wrong thing.

1c. Read `feedback.py`'s `get_critique`

This function is already finished and shows how the pieces connect:

```
def get_critique(image, style, area):
    instructions = get_improvement_instructions(area)      # Feature 3 (you write)
    prompt = build_prompt(style, area, instructions)       # Feature 4 (you write)
    ai_text, is_real = ask_ai_about_image(image, prompt)   # ai_helper.py (done)

    if not is_real:
        return ai_text
    return format_feedback(ai_text)                        # Feature 5 (you write)
```

It runs **four stages in a row**, and each stage's output feeds the next. Keep this picture in mind. Steps 4, 5 and 6 each build one stage.

> **CHECKPOINT**
> Can you answer these? (1) What does a handler function return to? (2) Which file talks to the AI? (3) Which three functions in `feedback.py` are still unfinished? If yes, move on.

Step 2: Feature 1, look up an art style

File: `art_styles.py` **Function:** `get_style_description(style_name)`

Goal: when the user picks a style in the dropdown, return its description.

Ideas you need: dictionaries, `if` statements, returning a value.

Understand the data first

Look at the top of `art_styles.py`. A **dictionary** stores pairs of *key* to *value*:

```
ART_STYLES = {
    "Impressionism": "Loose, visible brushstrokes ...",
    "Cubism": "Objects are broken into geometric shapes ...",
    ...
}
```

So `ART_STYLES["Cubism"]` gives you Cubism's description text. The helper `get_style_names()` (already written) returns just the keys, which is how the dropdown gets its choices.

TODO 1: handle "nothing chosen yet"

When the page first loads, or the user clears the dropdown, `style_name` is `None` (or an empty string). Both of those count as **falsy** in Python, so one check catches both:

```
if not style_name:
    return "Please choose a style."
```

Put this at the top of `get_style_description`, replacing nothing yet.

TODO 2: look up the style

There are three ways to look up a key. Try to predict what each does when the key is missing.

Option	Code	If the key is missing
A	`ART_STYLES[style_name]`	Crashes with a `KeyError`
B	`ART_STYLES.get(style_name, "Unknown style")`	Returns `"Unknown style"`
C	a long `if / elif / else` chain	Works, but you must add a new branch for every style

Option B is the shortest and the safest, so use it. The second argument to `.get()` is the "default" value returned when the key is not found.

TODO 3: return it

The result of the lookup is the description. Return it so Gradio can display it. Combine TODOs 2 and 3 on one line: `return ART_STYLES.get(style_name, "Unknown style")`.

Finished function

art_styles.py

```
def get_style_description(style_name):
    """Look up a style and return its description as text."""
    if not style_name:
        return "Please choose a style."
    return ART_STYLES.get(style_name, "Unknown style")
```

You may delete the old line `return "TODO: implement get_style_description() ..."`.

CHECKPOINT

Restart the app. In tab 1 choose **Cubism**. You should see the description about geometric shapes. Choose each style and read it. Now test your function without the browser: in the terminal, run `python -c "from art_styles import get_style_description as g; print(g('Pop Art')); print(g(None)); print(g('Nope'))"`. You should see the Pop Art description, then `Please choose a style.`, then `Unknown style`.

STUCK?

If you still see the TODO message, you either did not save the file or did not restart the app. If you get `IndentationError`, make sure the lines inside the function are indented by four spaces. If you see `KeyError`, you used `ART_STYLES[style_name]` without `.get`.

Step 3: Feature 2, report the size of the upload

File: `app.py` **Function:** `handle_upload(image)`

Goal: when the user uploads artwork, tell them its size in pixels.

Ideas you need: objects and attributes, f-strings.

How the image travels

Read the comment block at the top of `handle_upload`. The component is `gr.Image(type="pil", ...)`. That `type="pil"` setting tells Gradio to hand your function a **PIL Image object** (PIL is the Pillow library). If nothing is uploaded, `image` is `None`. That is why the function checks `if image is None` first. **Always check for `None` before using an input**, or your code will crash on `None.width`.

TODO 1: add the size

A PIL image has two attributes: `image.width` and `image.height`. Use an **f-string** (a string that starts with `f` and lets you put variables inside `{ }`) to build the message:

```
return f"Artwork received! Size: {image.width} x {image.height} pixels."
```

Finished function

app.py

```
def handle_upload(image):
    # HOW THE IMAGE TRAVELS:
    #   1. The user drops a file into gr.Image in the browser.
    #   2. Gradio uploads it and, because we set type="pil", converts it to a
    #      PIL Image object.
    #   3. Gradio calls this function and passes that object as `image`.
    #   4. If nothing is uploaded, `image` is None.
    #
    # TODO 1 (optional): Also report the image size.
    #         PIL images have .width and .height, e.g. f"{image.width}x{image.height}"
    if image is None:
        return "No artwork uploaded yet."
    return f"Artwork received! Size: {image.width} x {image.height} pixels."
```

CHECKPOINT

Restart, go to tab 2, and upload any picture. The Status box should say something like `Artwork received! Size: 640 x 480 pixels.` Check that the numbers match your file's properties. Then click the small **x** to remove the image; you should see `No artwork uploaded yet.`

NOTE

Notice the upload component `image_input` is defined in tab 2 but **used in tab 3** (in the `inputs=[image_input, ...]` list). Components are just Python variables, so any tab can use them.

Step 4: Feature 3, instructions for each improvement area

File: `feedback.py` **Function:** `get_improvement_instructions(area)`

Goal: the user picks one of five areas (`IMPROVEMENT_AREAS`). Return **one sentence** that tells the AI what to focus on for that area.

Ideas you need: `if / elif / else` and comparing strings with `==` .

Why is this a separate function?

A good prompt is specific. "Give feedback" gets vague answers. "Focus on how light and shadow create form and depth" gets useful ones. This function holds the specific sentence for each area so `build_prompt` (next step) can simply plug it in.

TODO 1 and TODO 2: one branch per area

Each area gets its own branch. The starter text gives you Shading. Remember that `==` compares and `=` assigns. Add `elif` branches for Composition, Color and Proportions. Ask yourself, for each one: *what would a real art teacher look at?*

- **Composition**: arrangement, balance, focal point.
- **Color**: harmony, contrast, mood.
- **Proportions**: relative sizes of shapes and body parts.

TODO 3: the `else`

`else` catches everything that did not match, which here means "General feedback" (or any unexpected value, which keeps your function safe). Return a sentence covering the whole piece.

Finished function

feedback.py

```
def get_improvement_instructions(area):
    """Return a sentence telling the AI what to focus on."""
    if area == "Shading":
        return "Focus on how light and shadow create form and depth."
    elif area == "Composition":
        return ("Focus on how the subjects are arranged in the frame: balance, "
                "focal point, and where the eye travels.")
    elif area == "Color":
        return ("Focus on the color choices: harmony, contrast, and whether the "
                "palette fits the mood.")
    elif area == "Proportions":
        return ("Focus on the relative size and placement of shapes and body "
                "parts: do they look believable?")
    else:
        return "Give feedback on the whole piece: shading, composition, color, and proportions."
```

Your wording can be different. That's fine and even good. It only matters that each area returns a **different, specific** sentence.

CHECKPOINT

In the terminal run `python -c "from feedback import get_improvement_instructions as g; print(g('Color')); print(g('General feedback'))"` and check you get two different sentences. Then try `g('Banana')` and confirm it falls into the `else` without crashing.

STUCK?

`SyntaxError: invalid syntax` on an `if` line usually means a missing colon `:` at the end. `SyntaxError: 'elif'` ... usually means an `elif` is not lined up with its `if`. All of `if`, `elif` and `else` must start at the same indent.

Step 5: Feature 4, build the AI prompt

File: `feedback.py` **Function:** `build_prompt(style, area, instructions)`

Goal: assemble one string, the **prompt**, that tells the AI who it is, what the student wants, and exactly how to format its answer.

Ideas you need: string concatenation, f-strings, and `\n` (a newline character).

What is a prompt?

The AI knows nothing about your student except what you write in the prompt. Everything it needs (the role, the target style, the focus area, the answer format) must be in that one string. Building good prompts is a real skill, and it is mostly just careful writing.

TODO 1: start with a role

```
prompt = "You are a kind, encouraging art teacher.\n\n"
```

`\n` means "new line", and `\n\n` leaves a blank line. A role changes the tone of the answer.

TODO 2: mention the style and area

Use f-strings to insert the function's parameters. Use `+=` to **add onto** the end of `prompt`:

```
prompt += f"The student wants to move toward the {style} style.\n"
prompt += f"They would like help with: {area}.\n"
```

TODO 3: add the instructions

`instructions` is the sentence from Step 4. Add it on:

```
prompt += instructions + "\n\n"
```

TODO 4: ask for three labelled sections

This is the most important line for the next step. **Feature 5 will split the AI's answer on these exact labels**, so spell them exactly: `STRENGTHS:` , `AREAS TO IMPROVE:` , `EXERCISES:` .

```
prompt += (
    "Reply in exactly three labelled sections, in this order:\n"
    "STRENGTHS:\n"
    "AREAS TO IMPROVE:\n"
    "EXERCISES:\n"
    "Keep each section short and use bullet points."
)
```

Python joins adjacent string literals inside parentheses into one string. Notice there are no commas between them.

TODO 5: return the prompt

End with `return prompt` .

Finished function

feedback.py

```
def build_prompt(style, area, instructions):
    """Build the full text prompt that is sent to the AI along with the image."""
    prompt = "You are a kind, encouraging art teacher.\n\n"
    prompt += f"The student wants to move toward the {style} style.\n"
    prompt += f"They would like help with: {area}.\n"
    prompt += instructions + "\n\n"
    prompt += (
        "Reply in exactly three labelled sections, in this order:\n"
        "STRENGTHS:\n"
        "AREAS TO IMPROVE:\n"
        "EXERCISES:\n"
        "Keep each section short and use bullet points."
    )
    return prompt
```

NOTE

Checkpoint (demo mode): Restart the app. Upload an image in tab 2. In tab 3 choose the style **Cubism**, the area **Color**, and click **Get feedback**. In demo mode the output shows your prompt at the bottom. It should look like this:

```
You are a kind, encouraging art teacher.

The student wants to move toward the Cubism style.
They would like help with: Color.
Focus on the color choices: harmony, contrast, and whether the palette fits the mood.

Reply in exactly three labelled sections, in this order:
STRENGTHS:
AREAS TO IMPROVE:
EXERCISES:
Keep each section short and use bullet points.
```

Change the style and area and click again. The prompt text should change to match. If it does not, check you used `{style}` and `{area}` inside an f-string (the string must start with `f`).

WHY

Reading the prompt in demo mode is like seeing what the AI sees. If something is missing from the prompt, the AI cannot possibly use it. Always debug prompts by printing them first.

STUCK?

If you see the literal text `{style}` in the output, you forgot the `f` before the opening quote. If the prompt is missing the area, check you returned `prompt` and not a shorter variable.

Step 6: Feature 5, format the AI's answer

File: `feedback.py` **Function:** `format_feedback(ai_text)`

Goal: the AI answers with the three labelled sections. Split them apart and display them as clean headings and lists.

Ideas you need: `str.split`, `str.strip`, `in`, tuple-style unpacking, `if`.

What the AI text looks like

Because of your prompt, the reply should look something like this:

```
STRENGTHS:
- Nice use of contrast in the sky
AREAS TO IMPROVE:
- The hand looks too small compared with the head
EXERCISES:
1. Draw 10 hands from photos, 2 minutes each
```

And this is what we want the user to see:

```
## Strengths
- Nice use of contrast in the sky

## Areas to Improve
- The hand looks too small compared with the head

## Suggested Exercises
1. Draw 10 hands from photos, 2 minutes each
```

(`##` makes a heading, because the output box is a Markdown component. You can add emoji to the headings like the target in the starter file if you wish.)

TODO 1: split into three pieces

`text.split(separator, 1)` cuts a string at the **first** occurrence of the separator and returns a list of two parts. We cut three times, once at each label, and unpack the two-item list into two variables:

```
before, rest = ai_text.split("STRENGTHS:", 1)
strengths, rest = rest.split("AREAS TO IMPROVE:", 1)
improvements, exercises = rest.split("EXERCISES:", 1)
```

Walk through it with the example above: after line 1, `rest` is everything after `STRENGTHS:`. After line 2, `strengths` is the text between the first two labels and `rest` is whatever follows `AREAS TO IMPROVE:`. After line 3 you have the other two pieces. (`before` is anything the AI wrote before the first label; we ignore it.)

TODO 2: strip extra whitespace

The pieces still have blank lines at the start or end. `.strip()` removes whitespace from both ends: `strengths.strip()`.

TODO 3: build the output string, and handle a missing section

The bonus in the starter file asks: *what if the AI forgot a section?* If a label is missing, `.split(label, 1)` returns only **one** part, and unpacking into two variables crashes with a `ValueError`. So check **before** splitting, and if anything is missing return the raw text unchanged:

```
if ("STRENGTHS:" not in ai_text or "AREAS TO IMPROVE:" not in ai_text
        or "EXERCISES:" not in ai_text):
    return ai_text
```

The line break is allowed because everything inside the parentheses belongs to one `if`.

Finished function

feedback.py

```
def format_feedback(ai_text):
    """Organize the AI's raw answer into strengths / improvements / exercises."""
    # If the AI ignored our format, show its answer untouched.
    if ("STRENGTHS:" not in ai_text or "AREAS TO IMPROVE:" not in ai_text
            or "EXERCISES:" not in ai_text):
        return ai_text

    before, rest = ai_text.split("STRENGTHS:", 1)
    strengths, rest = rest.split("AREAS TO IMPROVE:", 1)
    improvements, exercises = rest.split("EXERCISES:", 1)

    return (
        "## Strengths\n" + strengths.strip() + "\n\n"
        "## Areas to Improve\n" + improvements.strip() + "\n\n"
        "## Suggested Exercises\n" + exercises.strip()
    )
```

Test it without any API key

You can test `format_feedback` on its own by giving it fake AI text. Run this in the terminal:

```
python -c "from feedback import format_feedback as f; print(f('STRENGTHS:\n- good line\nAREAS TO
    IMPROVE:\n- hands\nEXERCISES:\n1. draw hands'))"
```

You should see three headed sections. Then try `f('The AI ignored my format')` and check you get the same sentence back untouched. Good code deals with surprises gracefully.

Bonus: when the AI uses bold headings

Models often write `**STRENGTHS:**` in bold. After your split, a stray `**` is left over at the end of one section and the start of the next. Fix it with a tiny helper, and use it in place of `.strip()`:

feedback.py

```
def clean_section(section):
    """Strip whitespace and any stray ** left over from bold section headings."""
    return section.strip().strip("*").strip()
```

Then write `clean_section(strengths)`, `clean_section(improvements)` and `clean_section(exercises)` inside `format_feedback`.

> **CHECKPOINT**
> There is nothing new to see in demo mode, because demo and error messages are shown as-is (look at the `if not is_real` line in `get_critique`). You will see your formatting in real use in the next step. Make sure the two terminal tests above passed first.

STUCK?

`ValueError: not enough values to unpack` means a label was missing from the text before you split. Add the `if ... not in ai_text` guard. If sections run together, check that you added `"\n\n"` between them.

Step 7: Turn on the real AI and test everything

Goal: see your whole pipeline working with real feedback.

1. Get a Gemini API key from your teacher (or create one at aistudio.google.com/apikey).
2. Open `.env` (the file you copied from `.env.example`) and set the key with no quotes and no spaces:

```
GEMINI_API_KEY=your-key-here
GEMINI_MODEL=gemini-3.5-flash
```

3. Stop the app (`Ctrl + C`) and run `python app.py` again. The yellow demo-mode banner at the top should be **gone**.
4. Upload a real drawing, choose a target style and an area, and click **Get feedback**. This takes a few seconds.

You should get a formatted critique with **Strengths**, **Areas to Improve** and **Suggested Exercises**.

Run your final test plan

Test	What you should see
Pick each of the 4 styles in tab 1	Four different descriptions
Upload a picture, then check tab 2	Its width and height
Click Get feedback with no picture	"Please upload artwork in the 'Upload Artwork' tab first."
Get feedback with no style chosen	"Please choose a target art style."
Same picture, 3 different areas	Noticeably different advice each time
Same picture, 2 different target styles	Advice that mentions each style

STUCK?
If you see `The AI request failed: ...`, read the end of the message. `API key not valid` or `403` means the key is wrong or has a space in it. `model` errors mean `GEMINI_MODEL` has a typo. If the banner still shows, `.env` must be in the same folder as `app.py` and the app must have been restarted.

Step 8: Optional extensions

Finished early? Each of these is a real feature you can add. Do them in any order.

1. **More styles.** Add Surrealism, Ukiyo-e, Art Nouveau or your own favorites to `ART_STYLES`. Nothing else needs to change, because the dropdown reads the dictionary's keys.
2. **Richer styles.** Turn each value into a dictionary such as `{"description": "...", "artists": ["Monet", "Renoir"]}`. Then update `get_style_description()` to print the artists too. (You will need to change how the lookup result is used. Think about what `ART_STYLES.get(...)` returns now.)
3. **Image details.** Show the image format and mode in `handle_upload`: `image.format` and `image.mode`.
4. **Skill level.** Add a `gr.Radio` for beginner or advanced in `app.py`, pass it through `handle_critique` and `get_critique`, and add a sentence to the prompt such as "Explain things simply for a beginner."
5. **History.** Use a `gr.State([])` to keep a list of past critiques and show them in a new tab.
6. **Compare two drawings.** A new tab with two `gr.Image` inputs and a prompt that asks the AI to compare them.

What you learned

- The **event, function, inputs, outputs** pattern that every Gradio app is built on.
- **Dictionaries** and safe lookups with `.get()`.
- `if / elif / else` to make decisions on text.
- **String building** with f-strings, `+=` and `\n`.
- **Splitting and cleaning text** with `split`, `strip` and `in`.
- That a **prompt is code**: precise instructions produce precise results, and you can test one with no AI at all.